

Chargement, inspection, filtrage

Analyse des données — Séance 2

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

Où nous allons aujourd'hui

1. Charger un vrai fichier, avec ses vrais défauts.
2. Comprendre ce qu'il contient — c'est-à-dire lire sa documentation.
3. Sélectionner, filtrer, trier.
4. Vérifier les types, comme promis la semaine dernière.

Le fil de la séance

Un fichier de données ne se comprend pas tout seul. Sans sa documentation, un tableau de chiffres n'est qu'un tableau de chiffres.

Les bibliothèques

Ne pas réécrire ce qui existe

Une bibliothèque est du code écrit par d'autres, testé, documenté, et mis à disposition. On l'importe plutôt que de le réécrire.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

L'abréviation après `as` est une convention, pas une obligation — mais c'est une convention universelle. Écrivez `pd`, jamais `pandas` ni `p`.

Ces trois bibliothèques sont déjà installées sur Colab. Il n'y a rien à télécharger.

Quand une bibliothèque manque

```
import statsmodels.api as sm
```

```
ModuleNotFoundError: No module named 'statsmodels'
```

Ce message ne dit pas que vous avez fait une faute de frappe. Il dit que la bibliothèque n'est pas installée.

```
!pip install statsmodels
```

Le ! en début de ligne indique à Colab qu'il s'agit d'une commande système, non de Python. Cela vous servira une fois ou deux dans le semestre.

Charger un fichier

Charger depuis une adresse, pas depuis votre disque

```
url = "https://www.data.gouv.fr/.../emissions.csv"  
df = pd.read_csv(url)
```

C'est la méthode à privilégier systématiquement. Le notebook devient **reproductible** : il s'exécute à l'identique chez votre binôme, chez votre correcteur, et sur votre machine dans six mois.

```
from google.colab import files  
fichiers = files.upload()
```

Le téléversement reste nécessaire pour les fichiers exigeant une inscription — de nombreuses données d'enquête. Il rend alors le notebook dépendant de votre disque, ce qui doit être signalé en commentaire.

Lire, et vérifier ce qu'on a lu

```
df = pd.read_csv("enquete_emploi.csv")

df.shape          # (nb de lignes, nb de colonnes)
df.columns        # les noms de variables
df.head()         # les 5 premières lignes
df.info()         # types et valeurs non manquantes
```

Ces quatre lignes se tapent **systématiquement**, avant tout calcul.

Un fichier chargé sans être inspecté est un fichier dont vous ignorez le contenu. La plupart des erreurs de ce cours viendront d'une inspection sautée.

Les fichiers français ne sont pas des fichiers anglais

```
df = pd.read_csv("donnees_insee.csv")           # une seule colonne ?
```

Trois conventions différentes, et il faut les déclarer :

```
df = pd.read_csv(
    "donnees_insee.csv",
    sep=";",           # séparateur point-virgule, pas virgule
    decimal=",",       # virgule décimale
    encoding="latin-1" # accents des fichiers anciens
)
```

Le symptôme d'un mauvais séparateur : `shape` annonce une seule colonne. Celui d'un mauvais encodage : des accents remplacés par des symboles.

```
df = pd.read_excel("dares_offres.xlsx",  
                  sheet_name="Donnees",  
                  skiprows=4)
```

Les tableaux publiés par les administrations comportent presque toujours un titre, une source et des notes autour des données. `skiprows` saute les lignes d'en-tête ; il faut d'abord **ouvrir le fichier et compter**.

Le réflexe

Avant de charger un fichier inconnu, ouvrez-le dans un tableur et regardez à quoi il ressemble. Trente secondes qui en économisent vingt minutes.

Comprendre ce qu'on a chargé

Un fichier d'enquête est accompagné d'un document qui décrit chaque variable : son nom, sa signification, ses modalités, son mode de collecte.

Sans ce document, le fichier est illisible. Avec lui, tout devient mécanique.

La compétence la plus sous-estimée du cours

Savoir chercher dans un dictionnaire des variables. Ce n'est pas une compétence informatique, et c'est celle qui distingue le plus nettement les étudiants en M1.

À quoi ressemble une entrée

Champ	Contenu
Nom	Code court, souvent obscur
Libellé	La question posée, ou le concept mesuré
Type	Quantitative, ou qualitative à modalités
Modalités	1 = ..., 2 = ..., 3 = ...
Champ	Sur qui la variable est-elle renseignée ?
Non-réponse	Comment est-elle codée ?

Les deux dernières lignes sont celles qu'on oublie de lire, et celles qui font les résultats faux.

Le champ de la variable

Une variable de durée du travail n'est renseignée que pour les actifs occupés. Calculer sa moyenne sur l'ensemble du fichier n'a aucun sens — et ne produit aucune erreur.

La non-réponse codée en chiffres

Une variable d'âge où « non renseigné » vaut 99. La moyenne se calcule sans broncher, et elle est fausse.

Il faut convertir ces codes en valeurs manquantes — c'est le sujet de la séance 3.

Aucun de ces deux problèmes n'est détectable sans la documentation.

Sélectionner, filtrer, trier

Choisir des colonnes

```
df["age"] # une colonne (Series)
df[["age", "sexe", "dipl"]] # plusieurs colonnes (DataFrame)
```

Les doubles crochets ne sont pas une coquetterie : vous passez une *liste* de noms. C'est la structure vue en séance 1.

```
df["statut"].value_counts()
df["statut"].value_counts(normalize=True)
```

`value_counts()` est le premier réflexe sur toute variable qualitative. Les effectifs qu'il affiche doivent correspondre aux modalités du dictionnaire — sinon, quelque chose ne va pas.

Filtrer des lignes

```
df[df["age"] >= 15]
```

La condition à l'intérieur produit une suite de vrai/faux, un par ligne. Le tableau ne conserve que les lignes vraies.

```
df[(df["age"] >= 15) & (df["age"] < 65)]           # et  
df[(df["statut"] == 1) | (df["statut"] == 2)]      # ou  
df[df["region"].isin(["11", "84", "93"])]          # appartenance
```

Les parenthèses autour de chaque condition sont obligatoires. Sans elles, Python évalue dans le mauvais ordre et renvoie une erreur peu explicite.

Le piège de l'indexation chaînée

```
df[df["age"] >= 15]["salaire"] = 0
```

SettingWithCopyWarning: A value **is** trying to be **set** on
a copy of a **slice from** a DataFrame

Deux crochets successifs produisent une *copie* : la modification s'applique à la copie, puis disparaît. Aucune erreur, aucun effet.

```
df.loc[df["age"] >= 15, "salaire"] = 0
```

.loc désigne lignes et colonnes en une seule opération. **Pour modifier, c'est toujours .loc.**

Chaîner plutôt qu'empiler

```
# Empile des variables intermediaires
df1 = df[df["age"] >= 15]
df2 = df1[["age", "salaire", "region"]]
df3 = df2.sort_values("salaire", ascending=False)

# Chaine : une seule expression, lisible de haut en bas
resultat = (
    df
    .loc[df["age"] >= 15, ["age", "salaire", "region"]]
    .sort_values("salaire", ascending=False)
)
```

Les parenthèses extérieures autorisent le retour à la ligne. C'est l'idiome courant de pandas : une opération par ligne, dans l'ordre où on les pense.

```
df.sort_values("salaire", ascending=False).head(10)
```

Trier par ordre décroissant et regarder les dix premières lignes est le moyen le plus rapide de repérer une anomalie : un salaire à sept chiffres, une année à 2099, un âge à 999.

```
len(df)                # nb de lignes  
df["statut"].isna().sum()  # nb de valeurs manquantes
```

Comptez avant et après chaque filtrage. Un filtre qui supprime les trois quarts de l'échantillon signale généralement une erreur de condition, pas un fait économique.

Les types, comme promis

Le code commune, épisode deux

```
df = pd.read_csv("communes.csv")  
df["CODGEO"].head()
```

```
0      1053  
1      1075
```

Le zéro initial a disparu : pandas a lu la colonne comme un nombre. Toute fusion avec un autre fichier échouera silencieusement.

```
df = pd.read_csv("communes.csv", dtype={"CODGEO": str})
```

La règle : **un identifiant n'est jamais un nombre**. Code commune, code NAF, SIREN, numéro de département — tous en texte.

Une colonne de nombres qui n'en est pas une

```
df["chomage"].mean()
```

```
TypeError: unsupported operand type(s)
```

`df.info()` révèle que la colonne est de type `object`, c'est-à-dire du texte. En cause : un espace comme séparateur de milliers, une virgule décimale, ou une modalité « nd » au milieu des chiffres.

```
df["chomage"] = pd.to_numeric(df["chomage"], errors="coerce")
```

`errors="coerce"` transforme en valeur manquante ce qui n'est pas convertible. Vérifiez alors combien de valeurs ont été perdues — si c'est beaucoup, le problème est ailleurs.

Trois vérifications avant de quitter le fichier

1. `df.shape` correspond-il à ce qu'annonce la documentation ?
2. `df.info()` : chaque colonne a-t-elle le type attendu ?
3. `value_counts()` sur les variables qualitatives : les modalités correspondent-elles au dictionnaire ?

Ces trois contrôles prennent deux minutes. Ils vous éviteront la moitié des erreurs du semestre — et ils constituent une question type du contrôle continu.

Pour la suite

Vous partirez de l'Enquête Emploi de l'INSEE, et vous reconstruirez le statut d'activité au sens du Bureau international du travail : actif occupé, chômeur, inactif.

- Retrouver dans la documentation les variables qui définissent ce statut
- Reconstruire les trois catégories
- Calculer un taux de chômage
- Le comparer au chiffre officiel, et expliquer l'écart

L'écart existera. Il ne viendra pas d'une erreur de code, mais de la définition retenue — population de référence, champ géographique, pondérations.

1. Terminez le chargement et l'inspection du fichier du TD.
2. Exécutez les trois vérifications sur vos données.
3. Repérez, sans les traiter, les variables qui contiennent des valeurs manquantes.
4. Récupérez le notebook de la séance 3 sur le site du cours.

`stefaniamarcassa.github.io/analyse_des_donnees`

Des questions ?